

PROJECT REPORT

University Cybersecurity Virtual Laboratory / Cyber Range

Why the platform is needed, how it was designed and implemented, which software and strategies were applied, and a complete solved question-and-answer guide to installation and configuration.

Container-first · Isolated student laboratories · Dynamic scheduling

Repository: <https://github.com/hussaini-8024/Project>

Management plane: FastAPI + React

Workload plane: Linux network namespaces, Docker, KVM/libvirt

Default private lab network: 10.0.0.0/8

This document describes a working campus Cyber Range as a Service. It is intended for instructors, examiners, and operators who must understand the motivation, the implementation path, and the exact install and configuration steps.

Contents

1. Why this project is needed
 2. Project objectives and scope
 3. How the system was built
 4. Software stack and tools
 5. Strategies and design principles
 6. Architecture and data flows
 7. Isolation, networking, and the private /8 range
 8. Capacity, quotas, and the scheduler
 9. Security controls and operations
 10. Solved questions and answers — installation and configuration
 11. Verification checklist and conclusion
- Appendix A. Demo accounts and default ports
- Appendix B. Environment variables

1. Why this project is needed

A university cybersecurity programme cannot be taught only from slides. Students must attack and defend real services: scan a target, exploit a deliberately vulnerable web application, inspect logs, and recover a host. Doing that on a shared classroom PC, on a student's personal laptop, or on the campus production network is unsafe. A misconfigured tool can harm another student, leak credentials, or touch systems that were never in scope.

Traditional computer labs also fail at scale. If every student boots a full Windows or Kali virtual machine, a single 128 GB server is exhausted after a handful of sessions. If every student shares one “hacking VLAN”, isolation disappears. If instructors hand out cloud credits, cost and governance become the bottleneck. The campus therefore needs a **private Cyber Range as a Service**: each student receives a persistent laboratory that looks like a small private network, while the university retains control of identity, quota, audit, and capacity.

The practical problems this project solves

- **Safety.** Vulnerable targets (DVWA, OWASP Juice Shop, WebGoat, Metasploitable) must never be reachable from another student or from the campus LAN.
- **Fairness.** One student must not consume the whole server. Quotas and a dynamic scheduler keep RAM, vCPU, disk, and running-container counts inside policy.
- **Pedagogy.** Instructors need assignments, prebuilt scenarios, snapshots, and a browser terminal so class time is spent on the exercise, not on hypervisor setup.
- **Operations.** Administrators need RBAC, MFA, audit logs, backups, ISO approval, and a Capacity Manager that measures safe concurrency instead of guessing “40–60 users”.
- **Efficiency.** Most coursework is Linux userspace. A container or a lightweight netns guest is enough. A full KVM virtual machine is used only when the kernel or a complete OS is required (Windows, kernel labs, some appliances).

Without this platform, the department either under-teaches (theory only), over-provisions (one fat VM per student), or accepts unsafe sharing. The Cyber Range is the missing middle: isolated enough for offensive labs, dense enough for a campus server, and operated like a service rather than a pile of scripts.

2. Project objectives and scope

The objective was to deliver a **production-shaped, multi-user university Cyber Range** with a clear split between the management plane (identity, UI, API, scheduler) and the workload plane (containers, VMs, student bridges). Students never receive the host Docker socket, a host root shell, or the libvirt management socket.

In scope	Out of scope / explicit non-goal
RBAC: super admin, administrator, instructor, lab manager, student	Guaranteeing a fixed 40–60 concurrent labs on every host
Persistent per-student laboratory with private 10.0.0.0/8 network	Letting students attack other students or the hypervisor
Container-first scheduling; KVM only when a full OS/kernel is required	Mounting docker.sock into student guests
Browser terminal and console gateways, assignments, quotas, audit, backups	Replacing the unrelated Ansible Office 2016 playbooks already in the repo
Staff-created extra networks and deploy-onto-network	Hard-coding a maximum user count instead of measuring capacity

Table 1. Scope of the Cyber Range platform.

3. How the system was built

The work was done as a monorepo. The management plane was written first so login, labs, machines, and quotas could be demonstrated without a hypervisor. Compute providers were then stacked behind one interface so the same API can run in development (mock or Linux namespaces) and in production (Docker + KVM hybrid).

3.1 Implementation sequence

- **Domain model.** SQLAlchemy models for users, sessions, quotas, student labs, networks, machines, templates, snapshots, assignments, audit logs, backups, and compute nodes.
- **Identity.** JWT access tokens, refresh sessions, password hashing (passlib/bcrypt), logout, optional TOTP MFA, role guards on every route.
- **Lab service.** First login creates a persistent StudentLab and a private LabNetwork (VLAN, namespace, bridge, CIDR 10.0.0.0/8).
- **Scheduler.** Container-first recommendation; quota checks; host thresholds (warning 85%, high/block 90%); queue when the lab RAM pool is insufficient.
- **Providers.** mock → namespace (Alpine rootfs + netns + veth + lab bridge) → docker → libvirt → hybrid. COMPUTE_PROVIDER=auto selects namespaces on a Linux host.
- **Guest runtime.** Real PTY over WebSocket: `sudo ip netns exec NS unshare --uts chroot MERGED /bin/busybox ash`. Same-lab ping works by IP and hostname.
- **Frontend.** React + TypeScript + Vite + Tailwind: dashboard, my lab, machine wizard, networks, topology, exercises, resources, admin people/infra/ops.
- **Packaging.** Docker Compose (Postgres, Redis, API, worker, nginx UI), Alembic, load-test CLI, deployment guide.
- **Admin networks.** Staff POST /api/networks and POST /api/networks/{id}/deploy so an administrator can create extra networks and place machines on them.

3.2 Repository layout

backend/	FastAPI app, providers, runtime, scheduler, Alembic
app/api/	REST: auth, users, labs, catalog, assignments, resources
app/providers/	mock, namespace, docker, libvirt, hybrid
app/runtime/	Alpine guests, netns/bridge/veth, PTY
app/services/	labs, scheduler, capacity, audit, guest, netutil
frontend/	React + TypeScript + Vite + Tailwind (port 5173)
docs/	Deployment, API, this report
loadtest/	Capacity runner (SAFE_* policy from live measurements)
docker-compose.yml	Postgres 16, Redis 7, API, worker, nginx UI
playbooks/	Pre-existing Ansible (unrelated to the range runtime)

3.3 How a student session actually runs

Alex Chen (STU-000245) signs in. The API restores laboratory LAB-2026-000245. The lab already has a private bridge and VLAN. Starting the Web Application Security scenario creates Kali, DVWA, and Juice Shop as containers (or namespace guests in auto mode). Each guest gets the next host address on 10.0.0.0/8 (gateway 10.0.0.1, first guest 10.0.0.2). The browser terminal attaches a PTY inside that guest. `ping dvwa-target` and `ping 10.0.0.3` succeed. Another student's lab uses the same RFC1918 numbers internally but a different namespace and bridge, so the packets never meet.

4. Software stack and tools

4.1 Management plane

Component	Software	Role in this project
API	Python 3.12, FastAPI, Uvicorn/Gunicorn	REST + WebSocket management plane
Validation	Pydantic v2 / pydantic-settings	Request bodies and .env configuration
ORM / DB	SQLAlchemy 2, Alembic	Schema and migrations
Dev database	SQLite (backend/data/cyberrange.db)	Zero-ops local development
Prod database	PostgreSQL 16	Multi-user durable state
Cache / queue	Redis 7, Celery worker	Session support and background jobs
Auth	python-jose (JWT HS256), passlib/bcrypt, pyotp	Sessions, hashing, TOTP MFA
UI	React 18, TypeScript, Vite 6, Tailwind 3	Operator and student console
UI data	TanStack Query, React Router 7	Client cache and routes
Terminal	xterm.js + Fit addon	Browser shell to the guest PTY
Topology	@xyflow/react	Lab network map
Charts	Recharts	Resource and capacity views
Reverse proxy	nginx 1.27	Static UI, /api and /ws reverse proxy, TLS termination

Table 2. Management-plane software.

4.2 Workload plane

Component	Software	When it is used
Default guests	Alpine miniroofts, busybox, Linux netns, veth, bridge	COMPUTE_PROVIDER=auto/namespace — real shells and intra-lab ping without Docker
Containers	Docker Engine	Production Linux services: DVWA, Juice Shop, WebGoat, Ubuntu, Wazuh
Full VMs	KVM, QEMU, libvirt	Windows, Metasploitable, kernel/full-OS labs
Images / disks	Shared container images, qcow2 backing files, CoW volumes	Density: many students share one golden image
Host metrics	psutil	Capacity Manager samples CPU, RAM, disk, IOPS
Packaging	Docker Compose, Python 3.12-slim and Node 22 Alpine images	Repeatable install

Table 3. Workload-plane software.

4.3 Exercise catalog (authorized training only)

Vulnerable systems are labeled **Training Target — Authorized Laboratory Use Only**. They are attached only to the owning student's isolated network.

Template	Kind	Purpose
Ubuntu / Debian service	Container	Linux administration and services
Kali Training	Container	Approved attacker workstation (nmap, Burp, Metasploit, ...)
DVWA, Juice Shop, WebGoat	Container	Authorized web-application security
Wazuh Monitor	Container	Blue-team detection labs
Metasploitable	VM	Classic vulnerable OS on the student VLAN
Windows Training	VM (KVM)	Full OS / kernel path

Template	Kind	Purpose
Prebuilt: web, network, Windows, Linux, defense	Bundle	One-click classroom scenarios

Table 4. Machine templates shipped with the range.

5. Strategies and design principles

5.1 Container-first, VM-when-required

The scheduler never prefers a full virtual machine when a container can run the exercise. A Linux userspace service shares the host kernel and a copy-on-write image. A Windows evaluation image or a kernel lab cannot. The HybridProvider sends containers to Docker and VMs to libvirt. This is the main density strategy: tens of web labs fit in the RAM that would otherwise hold a few fat VMs.

5.2 Plane separation

Students use HTTPS to nginx and the API. They never talk to Docker or libvirt. The workload plane holds the engines, student bridges, and disks. Operators SSH to the hypervisor; students do not. This is a classic management/control versus data/workload split.

5.3 Isolation by construction

Each lab has its own Linux network namespace, bridge, and VLAN id. Intra-lab traffic is bridged so classmates' machines in the same lab can ping. Inter-lab traffic is default-deny. Internet is off until staff enable SNAT. Host management addresses and Unix sockets are not in the student routing table.

5.4 Dynamic capacity, not a magic number

A 128 GB host with a 16–24 GB reserve can be an engineering target for roughly 40–60 container-heavy labs or 8–16 concurrent full VMs, but those figures are **not** coded as a hard cap. The Capacity Manager and `POST /api/resources/loadtest` measure CPU, RAM, storage, IOPS, and latency and print `SAFE_*` fields. Logged-in users are not the same as running containers. Hundreds of accounts can exist; only running workloads consume the lab RAM pool.

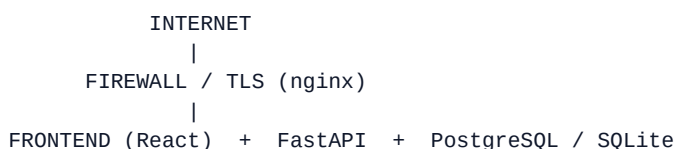
5.5 Least privilege and audit

RBAC is enforced in FastAPI dependencies (`require_staff`, `require_admin`). Students may only see their own machines and networks. Audit logs are append-only through the API. ISO uploads are checksummed and staff-approved before students can use them. Snapshots are counted against a per-student cap.

5.6 Provider strategy (dev → prod)

Development must work on a laptop. Production must use the real engines. One API, five providers: mock (UI without privileges), namespace (real shells on Linux), docker, libvirt, hybrid. Switching is an environment variable, not a rewrite.

6. Architecture and data flows



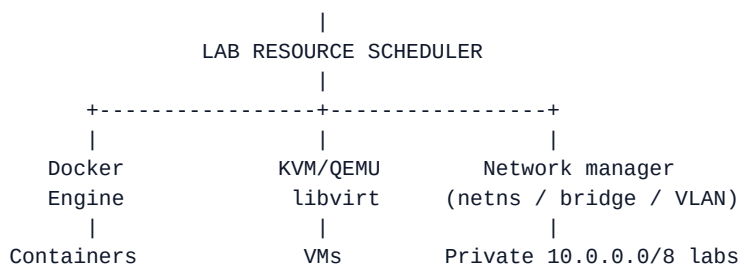


Figure 1. Management plane above, workload plane below.

Important API groups: `/api/auth` (login, refresh, MFA), `/api/labs` and `/api/machines` (lifecycle), `/api/networks` (list, staff create, staff deploy), `/api/assignments`, `/api/resources` (capacity and load test), `/api/audit` and `/api/backups`. WebSockets: monitoring, events, `/ws/terminal/{id}`, `/ws/console/{id}` with JWT in the query string and ownership re-checked.

7. Isolation, networking, and the private /8 range

Student laboratories use the RFC 1918 Class A network **10.0.0.0/8**, not a /24. The gateway is the first host (10.0.0.1). Guests are assigned 10.0.0.2, 10.0.0.3, and so on. Literal 0.0.0.0/8 is rejected because that block is IANA reserved. Administrators may create additional networks (prefix /8 through /30) and deploy catalog templates onto them. A student default network cannot be deleted; an empty admin-created network can.

Why /8 instead of /24? Classroom labs grow: Kali plus several targets plus staff-added machines must not run out of addresses, and operators think in “a private Class A lab”, not a tiny subnet. Isolation does not come from unique public CIDRs; it comes from per-lab namespaces and bridges. Two students can both use 10.0.0.2 without ever seeing each other.

On Linux, bridged ICMP can be stolen by iptables FORWARD unless `net.bridge.bridge-nf-call-iptables=0` (and the ip6/arp siblings) is set. The runtime applies that when it brings a lab bridge up. Overlayfs upperdirs are avoided on filesystems that cannot host them; each guest receives a small private copy of Alpine instead.

8. Capacity, quotas, and the scheduler

Profile	Containers	Running	RAM	vCPU	VMs	Storage	Snapshots
Basic	3	2	2048 MB	2	1	2 GB	2
Standard (default undergrad)	5	4	4096 MB	4	1	5 GB	3
Advanced (capstone)	10	8	8192 MB	6	2	20 GB	3

Table 5. Built-in quota profiles.

Host defaults assume 128 GB RAM (131072 MB) and 3.2 TB disk, with a 20 GB RAM reserve and 200 GB storage reserve for the OS, database, Redis, API, Docker, libvirt, and caches. The lab pool is total minus reserve. Thresholds: normal 80%, warning 85%, high/block 90%. Staff-initiated deploys onto a network are not blocked by the student’s remaining quota, so an administrator can still place a demonstration host in a full classroom lab.

9. Security controls and operations

- Students never receive `docker.sock`, `libvirt-sock`, or a host root shell.
- Login rate limit and lockout; optional administrator TOTP; HTTP-only cookies; CORS allow-list.
- Security headers: X-Content-Type-Options, X-Frame-Options DENY, Referrer-Policy same-origin.

- Internet default-deny; staff toggle per lab; SNAT only when enabled.
- Append-only audit of login, machine lifecycle, ISO, backup, network create/delete.
- Backups: database dump (14-day default retention), persistent volumes, not ephemeral exercise containers.
- ISO upload size cap and SHA-256; students see only approved images.
- Vulnerable templates carry an on-screen authorized-use warning.

10. Solved questions and answers — installation and configuration

This section is written as examination-style questions with complete, operational answers. Commands match the repository as shipped.

Q1. What are the two supported installation strategies, and when do you choose each one?

There are two first-class strategies.

A. Local development install — Python virtualenv + Node/Vite on the operator's machine. Use this to develop the UI/API, mark coursework, or demonstrate the range without standing up Postgres, Redis, or KVM. Database is SQLite. Compute provider is `auto` (Linux namespaces) or `mock`.

B. Packaged / production-shaped install — Docker Compose starts PostgreSQL 16, Redis 7, the FastAPI container, a Celery worker, and an nginx container that serves the built React app and proxies `/api` and `/ws`. On a real campus hypervisor you then set `COMPUTE_PROVIDER=hybrid` after Docker and libvirt are installed on the host. Choose A for coding; choose B for a repeatable service deploy. A third, host-native production path (section Q8) installs packages with apt and runs the same API under systemd/gunicorn behind nginx TLS.

Q2. What software must be present before a development install?

Python 3.12 or newer, Node.js 20+ (Node 22 is used in the frontend Docker image), npm, git, and a Linux kernel with network-namespaces support if you want real terminals. Optional but required for live guests: passwordless sudo for `ip netns`, `ip link`, `mount`, and `sysctl`. Windows/macOS development can still run the API/UI with `COMPUTE_PROVIDER=mock`, but intra-lab ping will be simulated.

Q3. Give the exact development installation commands.

From the repository root:

```
cd backend
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
mkdir -p data
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

Second terminal:

```
cd frontend
npm install
npm run dev
```

Open `http://localhost:5173` and sign in. Vite proxies `/api` and `/ws` to `127.0.0.1:8000` (see `frontend/vite.config.ts`). API docs: `http://localhost:8000/docs` Health: `http://localhost:8000/api/health`

Q4. What happens the first time the API starts?

Lifespan in `app/main.py` creates the storage directory, runs `Base.metadata.create_all`, migrates the networks table (adds `kind / created_by`, rebuilds away the old `UNIQUE(lab_id)` so a lab may have many networks), seeds demo users/templates/quotas if the database is empty, then rewrites leftover student /24 networks onto 10.0.0.0/8 and re-provisions running guests. You do not run a separate seed command.

Q5. Give the Docker Compose installation strategy end-to-end.

`cp .env.example .env` then edit secrets. `docker compose up --build`. Compose starts:

- postgres:16-alpine — user/password/db `cyberrange`, volume `pgdata`, healthcheck `pg_isready`
- redis:7-alpine — AOF persistence, volume `redisdata`
- api — backend/Dockerfile (python:3.12-slim), Uvicorn :8000, `DATABASE_URL` points at Postgres, `COOKIE_SECURE=true`
- worker — same image, `python -m app.worker`
- web — Node 22 build of the UI, `nginx:1.27-alpine`, ports 80 and 443

The API waits until Postgres is healthy. Persistent lab files live in volume `range-storage` mounted at `/app/data/storage`. Set `COMPUTE_PROVIDER` in `.env`; the Compose default is `mock` unless you override it.

Q6. Explain every important environment variable.

`APP_ENV=development|production|test`. `SECRET_KEY` — HMAC for JWT; generate with `openssl rand -hex 32`. `DATABASE_URL` — `sqlite:///./data/cyberrange.db` or `postgresql+psycopg2://cyberrange:PASSWORD@postgres:5432/cyberrange`. `REDIS_URL`. `CORS_ORIGINS` comma list (dev: `http://localhost:5173`; prod: `https://range.university.edu`). `COMPUTE_PROVIDER=auto|mock|namespace|docker|libvirt|hybrid`. `HOST_TOTAL_RAM_MB=131072`, `HOST_RESERVE_RAM_MB=20480`, `HOST_TOTAL_STORAGE_GB=3276`. `THRESHOLD_WARNING=85`, `THRESHOLD_HIGH=90`, `THRESHOLD_BLOCK=90`. `COOKIE_SECURE=false` on `http://localhost`, `true` behind TLS. Also in Settings: `access_token_minutes=30`, `refresh_token_days=7`, `session_idle_minutes=60`, `iso_max_gb=20`, `snapshot_max_per_student=3`, `backup_retention_days=14`, `ephemeral_ttl_hours=8`, `rate_limit_login=8`, `lockout_minutes=15`, `docker_socket=unix:///var/run/docker.sock`, `libvirt_uri=qemu:///system`, `storage_root=./data/storage`.

Q7. How do you configure CORS, cookies, and TLS correctly?

Development: `CORS_ORIGINS` includes the Vite origin; `COOKIE_SECURE=false`; `SameSite=lax`. Production: `CORS_ORIGINS` is only the public HTTPS origin; `COOKIE_SECURE=true`; terminate TLS on nginx or Traefik; enable HSTS; do not expose Uvicorn on the public NIC. nginx must forward `/api/` and upgrade WebSockets on `/ws/` (already in `frontend/nginx.conf`: `proxy_http_version 1.1`, Upgrade and Connection headers). Set X-Forwarded-Proto so the API knows it is behind HTTPS.

Q8. Describe the native production host installation (Ubuntu 22.04/24.04 or Debian 12).

Target: 128 GB RAM, 3.2 TB disk, /dev/kvm present.

```
sudo apt update
sudo apt install -y qemu-kvm libvirt-daemon-system libvirt-clients
bridge-utils \
docker.io docker-compose-plugin nginx postgresql redis-server certbot
python3-certbot-nginx
sudo usermod -aG kvm,libvirt,docker $USER
```

Enable forwarding: `echo 'net.ipv4.ip_forward=1' | sudo tee /etc/sysctl.d/99-cyberange.conf && sudo sysctl --system`. Obtain a certificate with certbot. Point nginx at the UI build and proxy API/WS as in `frontend/nginx.conf`. Run the API with gunicorn/uvicorn under systemd. Set `COMPUTE_PROVIDER=hybrid`, `DATABASE_URL` to PostgreSQL, `COOKIE_SECURE=true`. Never mount `/var/run/docker.sock` or `/var/run/libvirt/libvirt-sock` into student containers.

Q9. What storage layout should operators create on a 3.2 TB server?

`/var/lib/cyberange/images` — shared container and VM templates (CoW backing files).
`/var/lib/cyberange/volumes` — persistent student data.
`/var/lib/cyberange/vms` — thin-provisioned qcow2.
`/var/lib/cyberange/isos` — administrator ISO repository.
`/var/lib/cyberange/backups` — database + metadata + selected disks.
`/var/log/cyberange` — rotated logs.

Apply filesystem quotas per category. Snapshot count is also enforced in software (`snapshot_max_per_student`). Point `storage_root` at this tree in production.

Q10. How is the private lab network configured, and how do you verify it?

Default CIDR is 10.0.0.0/8 (`app/services/netutil.py DEFAULT_LAB_CIDR`). Gateway 10.0.0.1, guests from 10.0.0.2. Each LabNetwork stores `vlan_id`, `namespace`, `bridge`, `isolated`, `internet`, `kind` (`student|admin`). Runtime: create bridge, disable bridge-nf iptables, add veth into the guest netns, assign `ip addr add 10.0.0.2/8 dev eth0`. Verify: student Lab page shows 10.0.0.0/8; terminal `ip -4 addr` shows /8; `ping 10.0.0.3` and `ping dvwa-target` succeed; a second student's namespace is a different `ip netns` and cannot reach the first. Staff create extra nets with `POST /api/networks` (body: `name`, `cidr`, optional `lab_id`, `isolated`, `internet`) and deploy with `POST /api/networks/{id}/deploy` (template_slug, name, optional `owner_id`).

Q11. A student can ping by IP but not by hostname. What is the configuration explanation?

Hostnames are written into the guest's `/etc/hosts` inside the chroot (merged Alpine rootfs), not into the hypervisor's `/etc/hosts`. The browser terminal runs `ip netns exec ... chroot MERGED busybox ash`, so names resolve. A raw `ip netns exec NS ping dvwa-target` without chroot uses the host resolver and fails. This is expected, not a routing bug.

Q12. Intra-lab ping fails with 100% loss even though addresses look correct. What do you check?

1) Both guests are RUNNING and share the same `network_id`. 2) `ip -4 addr` inside each guest shows the /8 prefix, not a leftover /24. 3) `sysctl net.bridge.bridge-nf-call-iptables` is 0 (the runtime sets this; if a host policy resets it, bridged ICMP is filtered). 4) nftables/iptables must not FORWARD-drop the lab bridge. 5) You are not trying to ping another student's lab — that is denied by design.

Q13. How do compute providers get selected, and how do you switch from lab demo to production?

get_provider() reads COMPUTE_PROVIDER. auto and namespace → NamespaceProvider (Alpine + netns). docker → DockerProvider. libvirt → LibvirtProvider. hybrid → HybridProvider (Docker for containers, libvirt for VMs). anything else → MockProvider. Development: auto. Compose demo: mock until the host is a hypervisor. Campus server: hybrid after KVM and Docker are healthy. Restart the API after changing the variable (Settings is lru_cached).

Q14. How is nginx configured in the Compose UI image?

frontend/nginx.conf listens on 80, serves /usr/share/nginx/html, SPA fallback try_files to index.html. location /api/ proxies to http://api:8000/api/. /docs and /openapi.json proxy to the API. /ws/ upgrades the connection. That is the entire public configuration strategy for the containerized UI: one origin, no CORS needed between UI and API inside the same host header.

Q15. How do you install and run the load-test / capacity strategy?

As an administrator: POST /api/resources/loadtest with optional {"steps":[10,20,...,100]}. Or from the repo: python loadtest/run.py --api http://127.0.0.1:8000 --user admin --password 'CyberRange!Admin2026'. Use the printed SAFE_* fields as the production concurrency policy. Re-run after hardware or golden-image changes. Do not hard-code 40–60 students in application code.

Q16. How are backups configured, and what should not be backed up?

POST /api/backups (admin) records a backup object; retention is backup_retention_days (14). Production playbook: pg_dump on a cron, rsync or zfs send of /var/lib/cyberange/volumes, copy-on-write VM snapshots, replicate the approved ISO/template catalog. Test restore quarterly (database, lab metadata, one VM disk). Do not back up ephemeral exercise containers — they are rebuilt from templates.

Q17. How do you configure RBAC and the demo users after install?

Roles: super_admin, administrator, instructor, lab_manager, student. Seeded accounts (change immediately on a production host): admin / CyberRange!Admin2026, instructor / CyberRange!Teach2026, labmanager / CyberRange!Lab2026, student / CyberRange!Stud2026 (STU-000245, lab LAB-2026-000245). Administrators create users via POST /api/users. Enable TOTP MFA for operators. Students cannot POST /api/networks (403 Staff access required).

Q18. A student hits “vCPU quota exceeded” or “Storage quota exceeded”. What is the configuration lever?

Quota profiles Basic/Standard/Advanced are rows in resource_quotas. PATCH /api/quotas/{id} or assign a higher profile on the user (quota_name). Running-container, RAM, vCPU, VM, storage, and snapshot caps are independent. Starting an existing machine must not double-count its disk (the scheduler passes existing=machine). Staff deploy onto a network uses ignore_quota so an instructor can still place a target in a full student lab.

Q19. How do you configure isolation firewall rules beyond what the app creates?

The app creates per-lab bridges and namespaces. Operators should still add nftables/iptables that drop: lab → other lab bridges, lab → host management addresses, lab → Docker/libvirt sockets. Internet, when staff-enabled, is SNAT only. Do not enable unrestricted forwarding on the hypervisor. SSH to the hypervisor is limited to operators; noVNC/SPICE if used is bound to localhost behind nginx.

Q20. How would you add a second compute node (multi-node strategy)?

The scheduler already picks a healthy ComputeNode by RAM, CPU, KVM/Docker flags, and load. The first generation seeds node-01 as controller+worker. To scale: insert another compute_nodes row, run a worker on that host with the same DATABASE_URL/REDIS_URL, and ensure the provider on that worker can reach local Docker/libvirt. No application rewrite is required for the first extra node. Keep one management plane; do not give students a path to the new node's sockets.

Q21. List the production hardening checklist after installation.

MFA on for all administrators; demo passwords rotated; SECRET_KEY replaced; ISO uploads limited and checksummed; student outbound internet default-deny; audit log shipped to a write-once store; host reserve and 90% emergency threshold confirmed; console gateway bound to localhost; SSH to the hypervisor limited to operators; CORS and cookie flags match HTTPS; Postgres not published to the internet; Redis bound to the compose network only.

Q22. How do you verify a complete configuration with a classroom scenario?

1) Login as student. 2) Open My Lab — CIDR 10.0.0.0/8. 3) Create Machine → prebuilt Web Application Security (or start existing Kali/DVWA/Juice Shop). 4) Open Kali terminal; ping 10.0.0.3 and ping dvwa-target. 5) Logout; login as admin; Networks → Create network (10.0.0.0/8) attached to LAB-2026-000245 → Deploy debian/ubuntu. 6) Confirm the new host appears on that network. 7) GET /api/health, GET /api/resources, POST /api/resources/loadtest. 8) Confirm a second student cannot see the first student's machines.

Q23. Which Python and Node packages are load-bearing, and why were they chosen?

FastAPI — async API and OpenAPI for free. Uvicorn — ASGI. SQLAlchemy 2 — typed mappings. Alembic — SQL migrations when SQLite create_all is not enough. python-jose + passlib — standard JWT/password stack. pyotp — MFA without an external IdP. psutil — host samples for the Capacity Manager. docker SDK — optional live containers. Redis/Celery — worker and future job fan-out. React/Vite/Tailwind — fast campus UI. xterm.js — real terminal. pytest — scheduler, netutil, schema rebuild, runtime spec tests. These are mainstream tools a university operator can hire for, not a custom framework.

Q24. What must never be configured, even if it would “make labs easier”?

Do not put students in group docker or libvirt. Do not share one bridge for the whole class. Do not allocate 100% of RAM to labs (keep HOST_RESERVE_RAM_MB). Do not publish Postgres or Redis. Do not disable isolation to “fix ping”. Do not leave demo passwords on a networked host. Do not back up ephemeral containers as if they were unique data. Do not treat logged-in count as running-lab count when sizing the server.

11. Verification checklist and conclusion

The platform exists because a university cannot teach offensive and defensive security on shared desktops or on the production network. It was built as a management plane (FastAPI + React) over a workload plane (namespaces, Docker, KVM), with container-first scheduling, per-student 10.0.0.0/8 laboratories, staff-created extra networks, quotas, audit, and a Capacity Manager that measures safe concurrency instead of inventing it.

After installation, success looks like this: students sign in, receive a private lab, run authorized targets, ping peers inside that lab only, and never touch the hypervisor. Administrators create networks, deploy machines, read audit logs, and re-run load tests when hardware changes. That is the whole point of the Cyber Range — a controlled place to practise, at campus scale.

Appendix A. Demo accounts and default ports

Role	Username	Password	Notes
Super administrator	admin	CyberRange!Admin2026	Rotate on any shared host
Instructor	instructor	CyberRange!Teach2026	Assignments and class labs
Lab manager	labmanager	CyberRange!Lab2026	Images, ISOs, capacity
Student	student	CyberRange!Stud2026	STU-000245 / LAB-2026-000245

Table 6. Seeded demo accounts.

Service	Port	Bind
Vite UI (development)	5173	0.0.0.0, proxies /api and /ws to 8000
FastAPI / Uvicorn	8000	0.0.0.0 in dev; localhost behind nginx in prod
nginx UI (Compose)	80 / 443	public HTTP(S)
PostgreSQL (Compose)	5432	internal compose network only
Redis (Compose)	6379	internal compose network only

Table 7. Default ports.

Appendix B. Environment variables (quick copy)

```
APP_ENV=development
SECRET_KEY=change-me-in-production-use-openssl-rand-hex-32
DATABASE_URL=sqlite:///./data/cyberrange.db
# DATABASE_URL=postgresql+psycopg2://cyberrange:change-me@postgres:5432/cyberrange
REDIS_URL=redis://localhost:6379/0
CORS_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
COMPUTE_PROVIDER=auto
HOST_TOTAL_RAM_MB=131072
HOST_RESERVE_RAM_MB=20480
HOST_TOTAL_STORAGE_GB=3276
THRESHOLD_WARNING=85
THRESHOLD_HIGH=90
THRESHOLD_BLOCK=90
COOKIE_SECURE=false
```

End of report. Source repository: <https://github.com/hussaini-8024/Project> — see also docs/DEPLOYMENT.md and docs/API.md.